Accept MTN Mobile Money in Any App — CamPay & USSD (No Business License Required)

A beginner‑friendly, copy‑paste guide to collecting **MTN Mobile Money (MoMo)** payments in **any** application — a shop, an event‑ticket page, a donation button, a SaaS subscription, a school‑fees portal, anything. It covers **two methods**:

1. **USSD / MoMoPay** — works with **no business license and no API approval**. Best for students, hobby projects, and MVPs.
2. **CamPay API** — automated payments with a real PIN popup. Needs a **registered business** for live money, but you can build and test everything in a free **sandbox**.

> This is a self‑contained how‑to. It is not tied to the marketplace. If you also need **escrow** (holding money until delivery), see the companion doc `PAYMENT_SYSTEM_GUIDE.md`.

---

0. Which method should I use?

	USSD / MoMoPay	**CamPay API**
Business license needed?	❌ No	✅ Yes (for live money)
API approval needed?	❌ No	✅ Yes (for live)
Real money in development?	✅ Yes (it's real dialing)	Sandbox only until approved
User experience	User dials a code + PIN	One PIN popup on the phone
Payment auto‑verified?	⚠️ No — you reconcile manually	✅ Yes — via webhook
Best for	Students, MVPs, no‑license projects	Registered businesses, scale

Rule of thumb: No business license? → **start with USSD**. You can add CamPay later without changing your app's structure (both plug into the same "record a payment" step).

---

1. How MTN MoMo actually moves money (mental model)

There are only two directions:

- **Collect (inbound):** pull money **from** a customer → into **your** MoMo account.
- **Disburse (outbound):** send money **from** your account → **to** someone.

This guide covers **both directions**: **collect** (getting paid — §2 USSD, §3 CamPay) and **disburse** (paying people out — §4). Use whichever your platform needs; many apps need both (e.g. take payments *and* send refunds/payouts).

Every payment, regardless of method, ends with the **same backend step**: *"record that order/invoice X was paid."* Keep that step provider‑agnostic and you can swap methods freely.

```
Customer ──(USSD dial OR CamPay PIN popup)──▶ Your MTN MoMo account
                                                        │
                                                        ▼
                                  Your backend: mark invoice "PAID", deliver the
thing
```

---

2. METHOD A — USSD / MoMoPay (no business license)

2.1 The idea

MTN lets anyone send money to a **MoMoPay merchant number** by dialing a USSD code:

```
*126*9*<merchantNumber>*<amount>#
```

- `<merchantNumber>` = the number that **receives** the money (your own MoMo number / MoMoPay code).

- `<amount>` = the amount in XAF (whole number).

When the customer dials this and enters their PIN, **real money** moves to your account instantly. Your app's job is simply to (a) make dialing effortless and (b) record the payment.

> Orange Money equivalent is `*150#` (no amount pre - fill).

2.2 Frontend — build the code and open the dialer

On a phone, you can open the dialer **with the code already typed** using a `tel:` link. The only trick: `#` must be URL - encoded as `%23`.

```
<!-- A simple "Pay 2000 FCFA" button on any page -->
<button id="payBtn">Pay 2000 FCFA with MTN MoMo</button>
<p id="hint"></p>

<script>
  const MERCHANT_NUMBER = "670000000";   // YOUR MoMo / MoMoPay number
  const AMOUNT = 2000;                   // XAF, whole number
  const ussdCode = `*126*9*${MERCHANT_NUMBER}*${AMOUNT}#`;

  const isMobile = /android|iphone|ipad|ipod/i.test(navigator.userAgent);

  document.getElementById("payBtn").onclick = () => {
    if (isMobile) {
      // Opens the phone dialer pre-filled. User taps CALL, enters PIN → paid.
      window.location.href = `tel:${ussdCode.replace("#", "%23")}`;
      document.getElementById("hint").textContent =
        "Dialer opened — tap CALL and enter your MoMo PIN to pay.";
    } else {
      // Desktop: show the code so they can dial it on their phone.
      document.getElementById("hint").textContent = `Dial this on your phone:
${ussdCode}`;
    }
  };
</script>
```

Desktop tip — a scannable QR. Show a QR that opens the dialer when scanned with a phone:

```
const qrUrl =
  `https://api.qrserver.com/v1/create-qr-code/?
```

```
size=240x240&data=${encodeURIComponent(`tel:${ussdCode}`)}`;
// <img src={qrUrl} alt="Scan to pay" />
```

This is exactly the pattern used in `src/pages/PaymentReview.tsx` of this project.

2.3 Backend — record the payment

Because USSD happens **outside** your app (on the phone network), your server cannot automatically *prove* the money arrived. You **record** the customer's claim and reconcile against your MoMo statement.

```
// Node / Express example — works the same in any backend.
import express from "express";
const app = express();
app.use(express.json());

const MERCHANT_NUMBER = process.env.MERCHANT_MOMO_NUMBER;

app.post("/api/pay/ussd", async (req, res) => {
  const { invoiceId, amount, customerPhone } = req.body;

  // Build the same code server-side (don't trust a client-sent amount blindly).
  const ussdCode = `*126*9*${MERCHANT_NUMBER}*${Math.round(amount)}#`;

  // Record a PENDING payment the customer says they made.
  const payment = {
    id: `PAY-${Date.now()}`,
    invoiceId,
    amount: Math.round(amount),
    method: "mtn-ussd",
    customerPhone,
    ussdCode,
    status: "pending_confirmation", // becomes "paid" after you verify the MoMo
statement
    reference: `USSD-${Date.now()}`,
    createdAt: new Date().toISOString(),
  };
  await db.savePayment(payment);          // your DB
  res.json({ success: true, payment });
});
```

Reconciliation options (pick one):

- **Manual (simplest):** an admin checks the MoMo SMS/statement and flips matching payments to `paid`. Fine for low volume, demos, and no - license projects.

- **MoMo statement match:** match by amount + time + sender number.
- **Upgrade to CamPay later** (Method B) for automatic verification.

> ⚠️ **Never deliver high‑value goods instantly on an unverified USSD payment.** For digital goods or anything reversible, confirm first. For in‑person handoff (e.g. campus pickup) the seller can confirm the SMS before handing over.

2.4 USSD — pros & cons

- ✅ No business license, no API keys, no approval. Works today.
- ✅ Real money, instant to your account.
- ⚠️ No automatic verification → manual reconciliation.
- ⚠️ The customer does the dialing (one extra step vs an API popup).

---

3. METHOD B — CamPay API (automated; business for live, sandbox for dev)

CamPay (https://campay.net) is an aggregator that triggers a **PIN popup** on the customer's phone and tells your server (via **webhook**) when it's paid — fully automated.

3.1 Get set up

1. Create an account at **https://www.campay.net** → open the dashboard.
2. **Create an application** → you receive **App Username**, **App Password**, and an **App ID** (and optionally a **permanent access token**).
3. **Development:** use the **sandbox** base URL `https://demo.campay.net/api` with test credentials — build and test with fake money.
4. **Live money:** requires a **registered business** (e.g. RCCM in Cameroon). Until then, develop in sandbox or use **Method A (USSD)** for real money.

3.2 Store the keys in `.env`

```
CAMPAY_BASE_URL=https://demo.campay.net/api   # live: https://campay.net/api
CAMPAY_USERNAME=your_app_username
CAMPAY_PASSWORD=your_app_password
CAMPAY_ACCESS_TOKEN=                          # optional permanent token
CAMPAY_FORCE_MOCK=true                        # true while developing with no creds
```

3.3 Get an access token

```
let cachedToken = "";
let cachedTokenExpiry = 0;

async function getCampayToken() {
  if (cachedToken && Date.now() < cachedTokenExpiry) return cachedToken;

  // A permanent token from the dashboard skips this call entirely.
  if (process.env.CAMPAY_ACCESS_TOKEN) return
process.env.CAMPAY_ACCESS_TOKEN.trim();

  const res = await fetch(`${process.env.CAMPAY_BASE_URL}/token/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      username: process.env.CAMPAY_USERNAME,
      password: process.env.CAMPAY_PASSWORD,
    }),
  });
  if (!res.ok) throw new Error(`CamPay auth failed (${res.status})`);

  const data = await res.json();
  cachedToken = data.token;
  cachedTokenExpiry = Date.now() + 44 * 60 * 1000; // tokens last ~45 min
  return cachedToken;
}
```

3.4 Charge the customer (collect)

```
const useMock = () => process.env.CAMPAY_FORCE_MOCK === "true";

// Phones must be E.164: "+2376XXXXXXXX"
const toE164 = (p) => {
  const digits = String(p).replace(/\D/g, "");
  return digits.startsWith("237") ? `+${digits}` : `+237${digits}`;
};

async function chargeMtnMomo({ amount, phone, reference, description }) {
  // Mock mode: pretend it succeeded so you can build the whole flow with no creds.
```

```
  if (useMock()) {
    return { status: "successful", reference: `MOCK-${Date.now()}`, simulated: true
};
  }

  const token = await getCampayToken();
  const res = await fetch(`${process.env.CAMPAY_BASE_URL}/collect/`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Token ${token}`
},
    body: JSON.stringify({
      amount: Math.round(amount),
      from: toE164(phone),
      description,
      external_reference: reference, // YOUR invoice id — you'll get this back in
the webhook
      method: "mtn-momo",
      channel: "mtn",
    }),
  });

  const data = await res.json();
  if (!res.ok) throw new Error(data?.message || `CamPay charge failed
(${res.status})`);
  // Status is usually "pending" first; final result arrives via webhook.
  return { status: String(data.status || "pending").toLowerCase(), reference:
data.reference, raw: data };
}
```

Endpoint that your "Pay" button calls:

```
app.post("/api/pay/campay", async (req, res) => {
  const { invoiceId, amount, phone } = req.body;
  try {
    const result = await chargeMtnMomo({
      amount, phone,
      reference: invoiceId,                    // ties the payment to your
invoice
      description: `Payment for ${invoiceId}`,
    });
    await db.savePayment({ invoiceId, amount, method: "campay", status:
result.status, reference: result.reference });
    res.json({ success: true, status: result.status }); // tell UI "check your
phone for the PIN prompt"
  } catch (e) {
    res.status(400).json({ error: e.message });
  }
});
```

3.5 Confirm with a webhook (the source of truth)

Never mark an invoice paid just because the frontend got a response. The customer might cancel the PIN prompt. CamPay calls your **webhook** with the final status — that's what flips the invoice to `paid`.

```
app.post("/api/webhooks/campay", async (req, res) => {
  const event = req.body; // { status, reference, external_reference, amount, ... }

  // TODO: verify the signature/secret from your CamPay dashboard before trusting
it.
  const invoiceId = event.external_reference;
  const status = String(event.status || "").toLowerCase();

  if (["successful", "success", "completed"].includes(status)) {
    await db.markInvoicePaid(invoiceId, event.reference);
    // → deliver the product / unlock the subscription / send the ticket here.
  } else if (["failed", "cancelled"].includes(status)) {
    await db.markInvoiceFailed(invoiceId);
  }

  res.json({ status: "ok" }); // always 200 so CamPay stops retrying
});
```

Set this URL (`https://yourdomain.com/api/webhooks/campay`) in the CamPay dashboard.

3.6 CamPay — pros & cons

- ✅ Automated PIN popup + webhook verification (no manual reconciliation).
- ✅ Sandbox lets you build/test for free.
- ⚠️ Live money needs a registered business.

---

4. Sending money OUT — disburse / payouts

So far we **collected** money. Many platforms also need to **send money out** to people's MoMo:

- a **refund** to a customer,
- a **payout/withdrawal** to a seller, freelancer, driver, or creator,
- **prize money**, **cashback**, **payroll**, **supplier payments**, **affiliate commissions**, etc.

This is called **disburse** (a.k.a. withdraw / payout / transfer). The direction is reversed: money leaves **your** account and lands in **theirs**.

4.1 The license reality (read first)

- **CamPay disburse needs a registered, *funded* business account.** You can only send out money your CamPay balance actually holds, and live disbursement is a business feature. In **sandbox / mock** you can build and test it for free.
- **There is no "no - license" automatic disburse.** Unlike collection (where USSD lets the *customer* dial), sending money to someone else automatically requires an API. **Without a business/API**, your only option is to **pay them manually** from your own MoMo and **record** it in your app (see §4.5). That's perfectly fine for small/manual operations.

4.2 Disburse with CamPay (full code)

It mirrors `collect` — same token, but the `/withdraw/` endpoint and `to` (recipient) instead of `from`.

```
async function disburseMtnMomo({ amount, phone, reference, description }) {
  // Mock mode: simulate a successful payout while developing (no creds / no money
moved).
  if (useMock()) {
    return { status: "successful", reference: `MOCK-WD-${Date.now()}`, simulated:
true };
  }

  const token = await getCampayToken();
  const res = await fetch(`${process.env.CAMPAY_BASE_URL}/withdraw/`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Token ${token}`
},
    body: JSON.stringify({
      amount: Math.round(amount),
      to: toE164(phone),                  // recipient's MoMo number
      description,
      external_reference: reference,      // YOUR payout id — echoed back in the
webhook
```

```
      method: "mtn-momo",
      channel: "mtn",
    }),
  });

  const data = await res.json();
  if (!res.ok) throw new Error(data?.message || `CamPay payout failed
(${res.status})`);
  return { status: String(data.status || "pending").toLowerCase(), reference:
data.reference, raw: data };
}
```

4.3 The payout endpoint — balance check + idempotency (critical)

Paying money **out** is riskier than taking it in: a bug can pay someone **twice**. Two rules:

1. **Check the recipient is actually owed the money** (their internal balance ≥ amount) before sending.
2. **Idempotency:** never process the same payout request twice. Use a unique `payoutId` and refuse duplicates.

```
app.post("/api/payout", requireAuth, async (req, res) => {
  const { recipientUserId, amount, phone, payoutId } = req.body; // payoutId =
unique, client-generated

  // 1) Idempotency — if we've seen this id, return the previous result instead of
paying again.
  const existing = await db.getPayout(payoutId);
  if (existing) return res.json({ success: true, payout: existing, deduped: true
});

  // 2) Balance check — only pay out what the user has earned/has available.
  const balance = await db.getAvailableBalance(recipientUserId);
  if (balance < amount) return res.status(400).json({ error: "Insufficient balance"
});

  // 3) Deduct FIRST (so a retry can't double-spend), then attempt the transfer.
  await db.decrementBalance(recipientUserId, amount);

  try {
    const result = await disburseMtnMomo({
      amount, phone,
      reference: payoutId,
      description: `Payout ${payoutId}`,
    });
    const payout = {
      id: payoutId, recipientUserId, amount, phone,
      status: result.status === "pending" ? "processing" : "completed",
```

```
      reference: result.reference, createdAt: new Date().toISOString(),
    };
    await db.savePayout(payout);
    res.json({ success: true, payout });
  } catch (e) {
    // Transfer failed — refund the internal balance so the user isn't
shortchanged.
    await db.incrementBalance(recipientUserId, amount);
    res.status(502).json({ error: e.message });
  }
});
```

4.4 Confirm the payout with a webhook

Like collection, the **final** payout result arrives via webhook. Mark it `completed` only then (a `pending` payout can still fail at the operator).

```
app.post("/api/webhooks/campay", async (req, res) => {
  const event = req.body;                       // { status, external_reference,
... }
  const status = String(event.status || "").toLowerCase();
  const ref = event.external_reference;

  // Reuse the same webhook for collect AND disburse; branch on what the ref
belongs to.
  if (await db.isPayout(ref)) {
    if (["successful", "success", "completed"].includes(status)) await
db.markPayoutCompleted(ref);
    else if (["failed", "cancelled"].includes(status)) {
      await db.markPayoutFailed(ref);
      await db.refundPayoutBalance(ref);        // give the money back internally
    }
  } else {
    // …collection handling (mark invoice paid) as in §3.5…
  }
  res.json({ status: "ok" });
});
```

4.5 No‑license alternative — manual payout

If you have no business/API, you can still run payouts **manually** and keep your app's books correct:

1. The app shows an admin the list of pending payouts (who, how much, which MoMo number).

2. The admin sends the money from their **own** MoMo (normal transfer or `*126#`).
3. The admin marks the payout **done** in the app, which records the reference and deducts the internal balance.

Same data model as the automated flow — you just replace the `disburseMtnMomo()` call with a human. You can upgrade to automated CamPay disburse later without changing anything else.

4.6 Disburse — pros & cons

- ✅ Automated, webhook - confirmed payouts (CamPay).
- ✅ Mock/sandbox lets you build it with no money.
- ⚠️ Live payouts need a **funded, registered** CamPay business account.
- ⚠️ No automatic no - license option — fall back to **manual payouts** (§4.5).
- 🔒 Always enforce **balance checks + idempotency** so you never double - pay.

---

5. Embedding the keys & keeping them safe

This applies to **both** methods.

1. **Put secrets in a `.env` file** at your project root; load it at server start:
   - Node: `import "dotenv/config";` (or `node --env-file=.env`)
   - Deno: `deno run --env-file=.env …`
   - Then read with `process.env.X` (Node) or `Deno.env.get("X")` (Deno).
2. **Add `.env` to `.gitignore`** — never commit real keys. Ship a `.env.example` with blank placeholders so others know what to fill.
3. **Keys live on the server only.** The browser must **never** see your CamPay password/token. The "Pay" button calls **your** backend; your backend talks to CamPay.
   - The only payment value safe to send to the browser is the **public merchant number** (for USSD).
4. **On hosting** (Vercel, Render, Railway, etc.) set the same variables in the platform's **Environment Variables** panel — not in the repo.
5. **Verify webhooks** with the signing secret from the dashboard before acting on them.

Minimal `.env.example`:

```
# USSD (Method A) — no license needed
MERCHANT_MOMO_NUMBER=

# CamPay (Method B)
CAMPAY_BASE_URL=https://demo.campay.net/api
CAMPAY_USERNAME=
CAMPAY_PASSWORD=
CAMPAY_ACCESS_TOKEN=
CAMPAY_FORCE_MOCK=true
```

---

6. Testing without real money

- **CamPay:** set `CAMPAY_FORCE_MOCK=true`. The `chargeMtnMomo()` function returns a fake `"successful"` result, so you can build and demo the **entire** flow — button → "paid" → deliver — with no credentials and no money. Flip it to `false` (with sandbox or live creds) when ready.
- **USSD:** test the UI by checking that the button opens the dialer with the right code on a phone, and that your `/api/pay/ussd` endpoint records a pending payment. (The actual money step is just a normal MoMo dial.)

---

7. Copy - paste integration checklist (any project)

1. ☐ Decide what you need: **collect** (get paid), **disburse** (pay out), or both.
2. ☐ Pick a collect method: **USSD** (no license) or **CamPay** (business/sandbox). You can ship USSD now and add CamPay later.
3. ☐ Add the env vars (§5) and `.gitignore` your `.env`.
4. ☐ **Collect — Frontend:** a "Pay with MoMo" button. - USSD → build `*126*9*number*amount#`, open `tel:` (mobile) / show code + QR (desktop). - CamPay → POST to your `/api/pay/campay`, then tell the user "approve the PIN prompt on your phone."
5. ☐ **Collect — Backend:** one endpoint per method that **records** the payment against an invoice id.

6. ☐ **Collect — Confirmation:** - USSD → manual/statement reconciliation flips `pending → paid`. - CamPay → webhook flips `pending → paid` automatically.
7. ☐ **Deliver the thing** (unlock content, send ticket, mark order paid) **only after** the status is `paid`.
8. ☐ **Disburse (if you pay people out):** track an internal balance, then a payout endpoint with **balance check + idempotency** (§4.3) → CamPay `disburse` (business) or **manual payout** (§4.5) → confirm via webhook.
9. ☐ Test everything in mock/sandbox, then go live.

---

8. Common errors & fixes

Symptom	Likely cause	Fix
`tel:` link doesn't pre - fill the `#`	`#` not encoded	Use `ussdCode.replace("#", "%23")`.
CamPay `401 Unauthorized`	Token expired / wrong creds	Refresh token; check `CAMPAY_USERNAME/PASSWORD`; confirm `Authorization: Token <token>` header.
CamPay `Invalid amount`	Decimal/format issue	Send a whole number (`Math.round(amount)`); some setups need it as a string.
Charge "succeeds" but money never confirmed	Trusting the frontend response	Wait for the **webhook**; only it is the source of truth.
Works in sandbox, fails live	Using demo URL/creds in production	Set `CAMPAY_BASE_URL=https://campay.net/api` + live creds.
Phone rejected	Not E.164	Format as `+2376XXXXXXXX`.
Payout sent twice	No idempotency	Key payouts by a unique `payoutId`; refuse duplicates (§4.3).

Symptom	Likely cause	Fix
Disburse fails "insufficient funds"	CamPay business balance empty	Top up your CamPay account; you can only send money you hold.

TL;DR

- **Collect, no business license?** Use **USSD**: `*126*9*<yourNumber>*<amount>#`, open it with a `tel:` link (encode `#` as `%23`), record the payment, reconcile manually. Real money, works today.
- **Collect, have/registering a business?** Use **CamPay**: get keys → `getToken()` → `POST /collect/` → confirm via **webhook**. Build it now in sandbox/mock; flip to live later.
- **Disburse (pay people out)? CamPay** `POST /withdraw/` (needs a funded business account) with **balance check + idempotency**, or do **manual payouts** if you have no license. Confirm via webhook.
- **Always:** keys in `.env` (server‑side only, git‑ignored), the browser only triggers your backend, deliver/pay **only after** status is confirmed, and **never double‑pay** (idempotency on payouts).